

Control de concurrencia

Ing. Ricardo Tillerio

¿Qué es?

CONTROL DE CONCURRENCIA

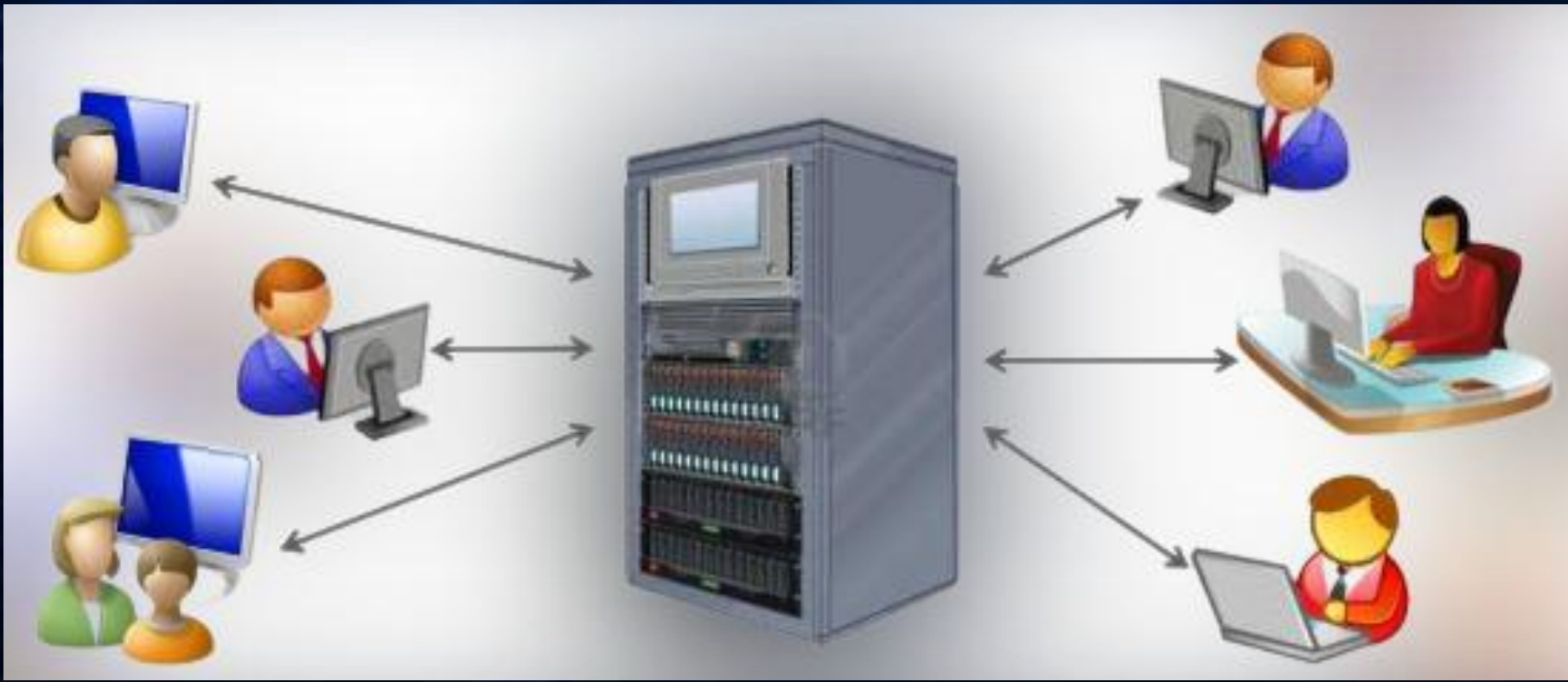
“El control de la concurrencia es la capacidad de administrar procesos simultáneos que acceden a la base de datos sin que interfieran entre sí”

- Garantizar la integridad de los datos.
- Aumentar la concurrencia en el acceso a los datos.
- Garantizar la consistencia de los datos.
- Establecer la seguridad de los datos.
- Controlar la redundancia de datos.
- Garantizar la recuperación de los datos.

CONTROL DE CONCURRENCIA

Problemas del acceso concurrente:

- Múltiples usuarios simultáneos del sistema de información provocan la ejecución de múltiples transacciones concurrentes.
- Si el aislamiento no está garantizado por el SMDB, puede haber problemas de consistencia.



CONTROL DE CONCURRENCIA

Problemas asociados a la concurrencia de transacciones:

- Problema de la actualización pérdida (**lost update**).
- Problema de la lectura sucia (**dirty read**).
- Problema del análisis inconsistente (**inconsistent analysis**).
- Problema de la lectura no repetible (**non-repeatable read**).
- Problema de la lectura fantasma (**phantom read**).

CONTROL DE CONCURRENCIA

Problema de la actualización perdida (**lost update**).

- Ocurre cuando dos transacciones entrelazadas, hacen actualizaciones sobre los mismos datos, sin esperar confirmación o cancelación de una de ellas, obteniendo actualizaciones erróneas de los datos.
- Unas transacciones sobrescriben las actualizaciones de otras.

Transacción 1	Transacción 2
$x_1 = \text{LEER}(X);$	
	$x_2 = \text{LEER}(X);$ $x_2 = x_2 + 1;$ ESCRIBIR (X, x_2); COMMIT ;
$x_1 = x_1 + 1;$ ESCRIBIR (X, x_1); COMMIT ;	



CONTROL DE CONCURRENCIA

Ejemplo de actualización perdida (**lost update**).

- Suponga que Jack y Jill tienen una cuenta de ahorros mancomunada con saldo de 1000 \$.
- Jill recibe su salario y decide depositar 100 \$ en la cuenta, en una sucursal cerca de su trabajo. Entre tanto, Jack necesita gastar algo de dinero y decide retirar 50 \$ de la cuenta, en una sucursal diferente.
- Si estas transacciones se ejecutaran de manera serial, una después de otra sin intercalación de sus operaciones, el balance final sería 1050 \$ sin que importara cuál se hubiera hecho primero.
- Sin embargo, si se realizan de manera concurrente el saldo final puede ser incorrecto.
- La siguiente figura muestra la programación de la ejecución concurrente que da como resultado un saldo de 1100 \$.

CONTROL DE CONCURRENCIA

Ejemplo de actualización perdida (**lost update**).

HORA	Transacción de Jack	Transacción de Jill	SALDO
t1	BEGIN TRANSACTION		
t2	read BAL(<i>lee 1000</i>)	BEGIN TRANSACTION	1000
t3	. . .	read BAL(<i>lee 1000</i>)	1000
t4	BAL = BAL – 50(<i>950</i>)	. . .	1000
t5	write BAL(<i>950</i>)	BAL = BAL + 100(<i>1100</i>)	950
t6	COMMIT	. . .	950
t7		write BAL (<i>1100</i>)	1100
t8		COMMMIT	1100

CONTROL DE CONCURRENCIA

Problema de la lectura sucia (**dirty read**).

- Es el problema más importante de todos.
- Supone que las transacciones en curso puedan leer datos que están siendo modificados por otra transacción que no ha finalizado y que podría cancelarse o fallar.

Transacción 1	Transacción 2
$x_1 = \text{LEER}(X);$ $x_1 = x_1 + 1;$ $\text{ESCRIBIR}(X, x_1);$	
	$x_2 = \text{LEER}(X);$ $\text{ESCRIBIR}(X, x_2);$ $\text{COMMIT};$
$\text{ROLLBACK};$	



Diagram illustrating a dirty read scenario. Transaction 1 updates variable X, but Transaction 2 reads the old value before Transaction 1 commits. Transaction 1 then rolls back, leaving Transaction 2 with stale data.

CONTROL DE CONCURRENCIA

Ejemplo de lectura sucia (**dirty read**).

- La siguiente figura muestra la secuencia de un ejemplo de actualización no comprometida que genera un error.
- La transacción INTERÉS calcula el interés para una cuenta de ahorro, con tasa de 5%, mientras que la transacción DEPÓSITO aporta 1000 \$ a la cuenta, pero ésta es deshecha.
- Suponga que el saldo inicial en la cuenta es de 1000 \$. Si DEPÓSITO actualizara en verdad el valor del saldo para que fuera 2000 \$ antes de que INTERÉS lo leyera, la cantidad que almacenaría INTERÉS sería 2100 \$.
- Sin embargo, DEPÓSITO se deshace, lo que significa que el estado de la base de datos no debe mostrar ningún efecto debido a DEPÓSITO, por lo que en el momento t_6 se restaura el valor original de 1000 \$ como resultado de deshacer DEPÓSITO.
- Observe que DEPÓSITO ignora que INTERÉS ha leído lo que escribió, e INTERÉS también ignora que se deshizo DEPÓSITO. La razón de deshacer DEPÓSITO pudo ser que la transacción era errónea, o era el resultado de una caída del sistema. En t_8 , se escribe el valor calculado por INTERÉS. Como INTERÉS se calculó con el saldo incorrecto, el valor final de éste también será incorrecto.

Ejemplo de lectura sucia (**dirty read**).

HORA	DEPÓSITO	INTERÉS	SALDO
	Transacción	Transacción	
t1	BEGIN TRANSACTION		1000
t2	read BAL (1000)	. . .	1000
t3	BAL = BAL + 1000(2000)	. . .	1000
t4	write BAL (2000)	BEGIN TRANSACTION	2000
t5	. . .	read BAL (2000)	2000
t6	. . .	BAL = BAL * 1.05(2100)	2000
t7	ROLLBACK	. . .	1000
t8	. . .	write BAL (2100)	2100
t9		COMMIT	2100

Problema del análisis inconsistente (**inconsistent analysis**).

- Ocurre cuando una transacción lee varios valores pero otra transacción actualiza algunos de ellos durante la ejecución de la otra.

Tiempo	Transacción 1	Transacción 2	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	Sum = 0	100	50	25	0
t ₃	read(bal _x)	read(bal _x)	100	50	25	0
t ₄	bal _x = bal _x - 10	sum = sum + bal _x	90	50	25	100
t ₅	write(bal _x)	read(bal _y)	90	50	25	100
t ₆	read(bal _z)	sum = sum + bal _y	90	50	25	150
t ₇	bal _z = bal _z + 10		90	50	35	150
t ₈	write(bal _z)		90	50	35	150
t ₉	commit	read(bal _z)	90	50	35	150
t ₁₀		sum = sum + bal _z	90	50	35	185
t ₁₁		commit	90	50	35	185

Valor viejo

Inconsistente

Inconsistente

CONTROL DE CONCURRENCIA

Ejemplo del análisis inconsistente (**inconsistent analysis**).

- Suponga que se desea encontrar el saldo total de todas las cuentas de ahorro pero se permite que durante esa transacción se realicen depósitos, retiros y transferencias.
- La figura de la transparencia siguiente contiene la secuencia que muestra el resultado incorrecto que pudiera producirse.
- La transacción SUMBAL calcula la suma del saldo de todas las cuentas. Para que el ejemplo sea pequeño se supondrá que sólo hay tres cuentas.
- Observe que esta transacción no modifica las cuentas sino sólo las lee.
- Al mismo tiempo, la transacción TRANSFER transfiere 1000 \$ de la cuenta A a la C; esta transacción se ejecuta y compromete correctamente. Sin embargo, interfiere con la transacción SUMBAL, que suma dos veces los 1000 dólares, una cuando está en la cuenta A y otra cuando llega a la cuenta C.

Ejemplo del análisis inconsistente (**inconsistent analysis**).

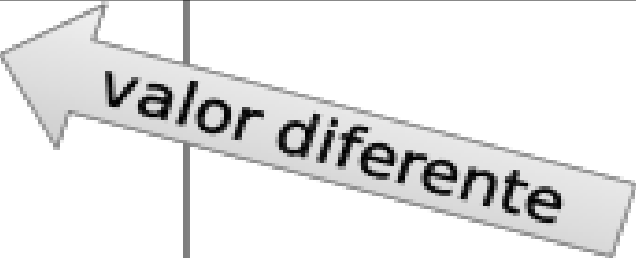
Hora	SUMBAL	TRANSFER	SALDO DE A	SALDO DE B	SALDO DE C	SUMA
t1	BEGIN TRANSACTION		5000	5000	5000	—
t2	SUM = 0;	BEGIN TRANSACTION	5000	5000	5000	—
t3	read BAL_A (5000)	. . .	5000	5000	5000	—
t4	SUM = SUM + BAL_A (5000)	read BAL_A (5000)	5000	5000	5000	—
t5	read BAL_B (5000)	BAL_A = BAL_A - 1000 (4000)	5000	5000	5000	—
t6	SUM = SUM + BAL_B (10000)	write BAL_A (4000)	4000	5000	5000	—
t7	. . .	read BAL_C (5000)	4000	5000	5000	—
t8		BAL_C = BAL_C + 1000 (6000)	4000	5000	5000	—
t9		write BAL_C (6000)	4000	5000	6000	—
t10	Read BAL_C (6000)	COMMIT	4000	5000	6000	—
t11	SUM = SUM + BAL_C (16000)		4000	5000	6000	—
t12	write SUM (16000)		4000	5000	6000	16000
t13	COMMIT		4000	5000	6000	16000

CONTROL DE CONCURRENCIA

Problema de la lectura no repetible (**non-repeatable read**).

- Se obtienen lecturas diferentes del mismo valor durante la misma transacción. El valor de los datos difiere con respecto a las lecturas anteriores.

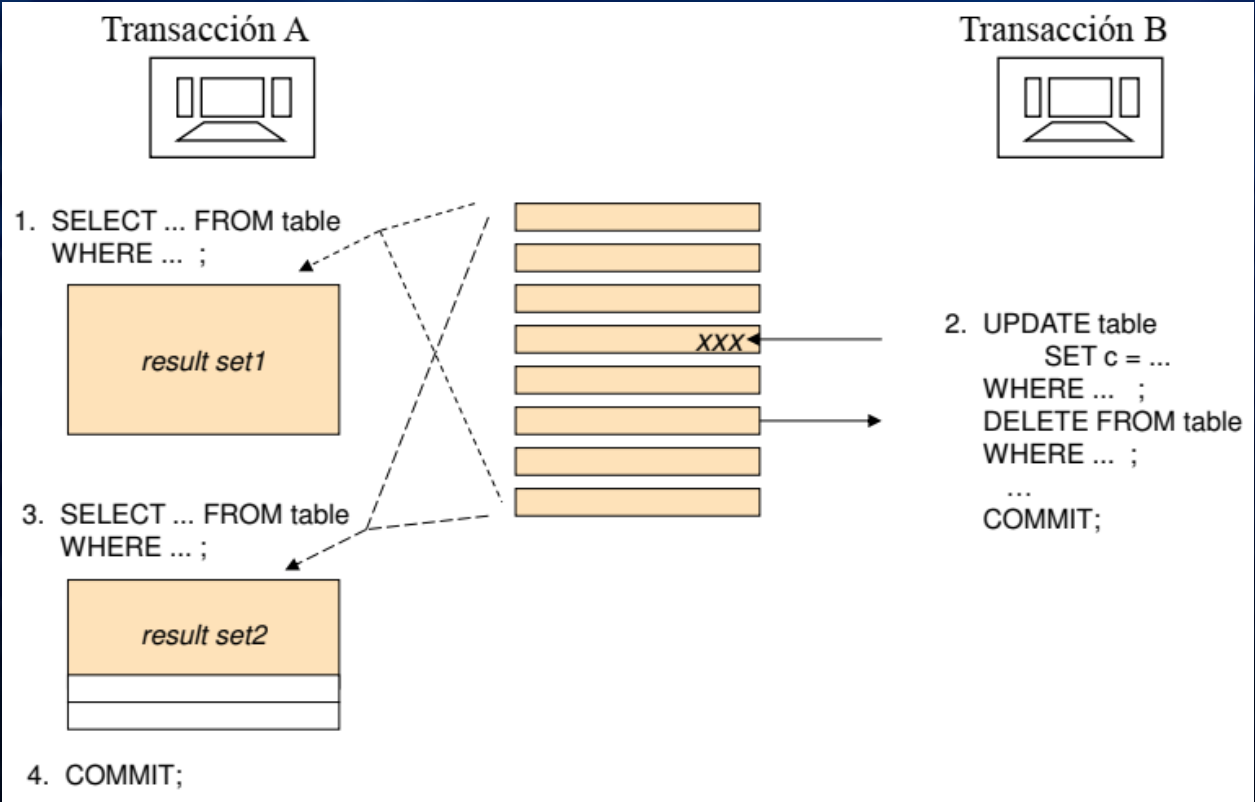
Transacción 1	Transacción 2
$x_1 = \text{LEER}(X);$ $\text{IMPRIMIR}(x_1);$	
	$x_2 = \text{LEER}(X);$ $x_2 = x_2 + 1;$ $\text{ESCRIBIR}(X, x_2);$ $\text{COMMIT};$
$x_1 = \text{LEER}(X);$ $\text{IMPRIMIR}(x_1);$ COMMIT	



CONTROL DE CONCURRENCIA

Ejemplo de lectura no repetible (**non-repeatable read**).

- Dos usuarios acceden la misma tiempo una tabla sin control de concurrencia. La anomalía se produce ya que los resultados de las consultas en la transacción A, no son estables, porque entre el tiempo en que se repiten, la transacción B actualiza la misma tabla, por lo que algunas de las filas previamente obtenidas podrían no tener los mismos datos o no estar disponibles.



CONTROL DE CONCURRENCIA

Problema de la lectura fantasma (phantom read).

- Sucede cuando una transacción en un momento lanza una consulta de selección con una condición y recibe N filas y posteriormente vuelve a lanzar la misma consulta y condición, y recibe N+n filas.
- Esto se debe a que otras transacciones insertaron nuevas filas que cumplen la condición impuesta en la consulta.

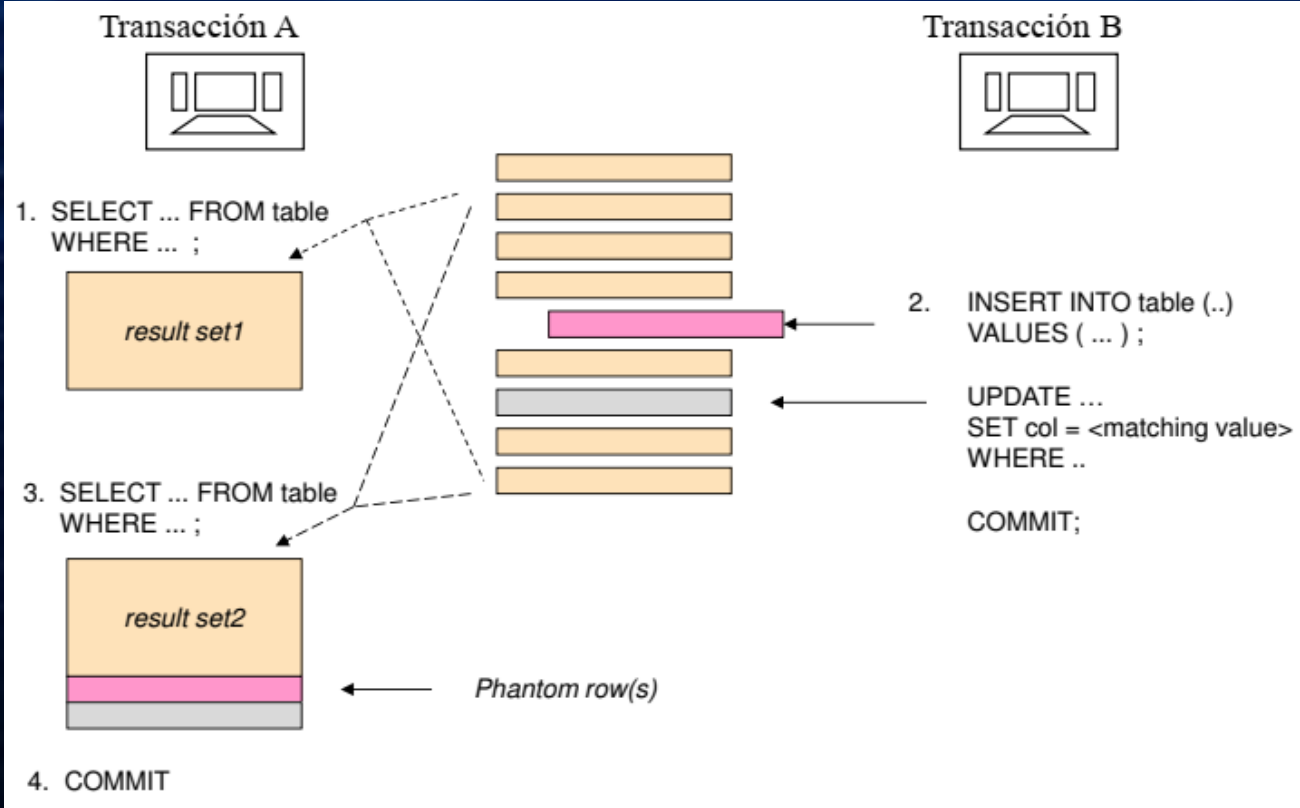
Transacción 1	Transacción 2
$\{x_i\} = \text{LEER}(\{X\})$; IMPRIMIR ($\{x_i\}$);	
	$\{x_j\} = \text{LEER}(\{X\})$; $\{x_j\} = \{x_j\} \cup \{x_k\}$; ESCRIBIR ($\{X\}, \{x_j\}$); COMMIT ;
$\{x_i\} = \text{LEER}(\{X\})$; IMPRIMIR ($\{x_i\}$); COMMIT	

una tupla (fantasma) más

CONTROL DE CONCURRENCIA

Ejemplo de lectura fantasma (phantom read).

- En el siguiente ejemplo, dos usuarios acceden la misma tiempo una tabla sin control de concurrencia. La anomalía se produce ya que los resultados de las consultas en la transacción A, no son estables, porque entre el tiempo en que se repiten, la transacción B actualiza la misma tabla incluyendo nuevas filas, así como modificando los valores de filas ya consultadas.



CONTROL DE CONCURRENCIA

¿Cómo evitar los problemas de concurrencia?

- Ejecutando las transacciones secuencialmente, serial o en serie:
 - $T1 \rightarrow T2 \rightarrow T3 \rightarrow \dots \rightarrow Tn$
 - Reduce mucho el rendimiento de un SGBD.
- Ejecutando las transacciones concurrentemente pero con un plan de ejecución secuenciable o serializable:
 - Debe tener el mismo efecto final que si las transacciones se ejecutaran secuencialmente (secuenciabilidad o serializable).
 - Se planificaría bajo mecanismos o técnicas de control de concurrencia tales como: bloqueos, marcas de tiempo (timestamps), control de concurrencia optimista o técnicas de validación, y versiones múltiples de los datos para garantizar la secuenciabilidad.

Niveles de aislamiento:

- El nivel de aislamiento especifica cómo se gestionan las transacciones que modifican la base de datos.
- Aseguran el aislamiento de transacciones.
- Cada motor relacional dispone de varias opciones de niveles de aislamiento.
- El nivel de aislamiento de una transacción determina el grado en el que otras transacciones simultáneas pueden afectar a los datos en los que opera.
- Los niveles de aislamiento ayudan a solventar los problemas asociados a la concurrencia de transacciones.

CONTROL DE CONCURRENCIA

ANSI SQL-92 define cuatro niveles de aislamiento (isolations levels). A mayor nivel de aislamiento mayor es el sacrificio con respecto a la concurrencia y al rendimiento. Niveles de menor a mayor aislamiento:

- **READ_UNCOMMITTED:** permite leer datos no confirmados de otras transacciones.
- **READ_COMMITED:** sólo permite leer datos confirmados antes de que se ejecuta una consulta. Suele ser el valor por defecto.
- **REPEATABLE_READ:** sólo permite leer datos confirmados antes de que comience la transacción.
- **SERIALIZABLE:** garantiza la secuenciabilidad de la transacción o que sea serializable con todas las demás transacciones concurrentes; si no puede conseguirlo, la transacción falla.

Configuración de los niveles de aislamiento:

- Se debe especificar al comienzo de la transacción:

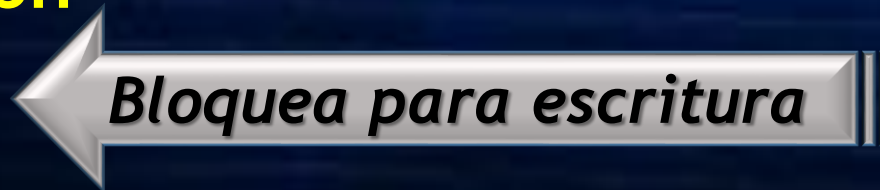
SET TRANSACTION ISOLATION LEVEL nivel;

- El nivel por defecto suele ser READ_COMMITTED, que ofrece un buen rendimiento.
- Para solucionar el problema de la actualización perdida con nivel READ_COMMITTED, algunos SGBD permiten bloquear para escritura al hacer un SELECT mediante la cláusula FOR UPDATE:

SELECT * FROM tabla

WHERE condición

FOR UPDATE;



CONTROL DE CONCURRENCIA

- A mayor nivel de aislamiento, menor rendimiento.

Nivel de aislamiento	Problemas de concurrencia			
	Lost update	Dirty read	Non-repeat. read	Phantom read
READ_UNCOMMITTED				
READ_COMMITTED				
REPEATABLE_READ				
SERIALIZABLE				

* Salvo que se use la cláusula FOR UPDATE

En MySQL: el nivel de aislamiento predeterminado REPEATABLE READ.

- El nivel de aislamiento actual se almacena en la variable del servidor `tx_isolation`.

```
1  mysql> SELECT @@tx_isolation;
2  +-----+
3  | @@tx_isolation |
4  +-----+
5  | REPEATABLE-READ |
6  +-----+
```

- La instrucción `SET TRANSACTION ISOLATION LEVEL`, cambia el nivel de aislamiento.

```
1  mysql> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
2
3  mysql> SELECT @@tx_isolation;
4  +-----+
5  | @@tx_isolation |
6  +-----+
7  | SERIALIZABLE   |
8  +-----+
```

Serialización de la planificación de transacciones:

- **Planificación serializable o secuencial de transacciones:** significa que las transacciones se ejecuta de manera concurrente, produciendo los mismos resultados que una ejecución de planificación serial.
- **Planificación de conflicto serializable de transacciones:** significa que una ejecución serializable de transacciones establece una secuencia de operaciones conflictivas.
- **Operaciones conflictivas:** dos operaciones son conflictivas si cumplen con todos los enunciados siguientes:
 - Pertenecen a transacciones diferentes.
 - Acceden al mismo ítem de datos.
 - Al menos una escribe en el mismo ítem.

Ejemplo de serialización de la planificación de transacciones:

- La figura siguiente muestra tres secuencias de planificación para las transacciones S y T, dos seriales y otra serializable.
- Las figuras (a) y (b) muestran la secuencias seriales (S,T) y (T,S), mientras que la figura (c) presenta una secuencia serializable que es equivalente en cuanto al conflicto a la primera secuencia serial (S,T).
- La lectura y escritura conflictivas de los ítem de datos ocurren en el mismo orden en la secuencia serializable que en la secuencia serial (S,T). Observe que en (a), S lee y escribe “x” antes de que T la lea y escriba, y S lee y escribe “y” antes de que lo haga T. En la secuencia serializable en (c) se preserva el mismo orden de lecturas y escrituras.
- También note que aquí no son equivalentes las dos secuencias seriales, pero ambas se consideran correctas y dejarían la base de datos en un estado consistente. Puede verificar que las dos secuencias seriales dan resultados diferentes, por medio de la asignación de valores iniciales a “x” y a “y”, y observar los valores finales que arrojan.

Ejemplo de serialización de la planificación de transacciones:

Hora	S	T	S	T	S	T
t1	BEGIN TRANS			BEGIN TRANS	BEGIN TRANS	
t2	read x			read x	read x	BEGIN TRANS
t3	$x = x * 3$			$x = x + 2$	$x = x * 3$	
t4	write x			write x	write x	
t5	read y			read y		read x
t6	$y = y + 2$			$y = y + 5$	read y	$x = x + 2$
t7	write y			write y	$y = y + 2$	write x
t8	COMMIT			COMMIT	write y	
t9		BEGIN TRANS	BEGIN TRANS		COMMIT	read y
t10		read x	read x			$y = y + 5$
t11		$x = x + 2$	$x = x * 3$			write y
t12		write x	write x			COMMIT
t13		read y	read y			
t14		$y = y + 5$	$y = y + 2$			
t15		write y	write y			
t16		COMMIT	COMMIT			
	(a) En serie: S, T		(b) En serie: T, S		(c) serializable	

Serialización

CONTROL DE CONCURRENCIA

Ejemplo de serialización exitosa y no exitosa:

	<u>Planificación Serie Seriarizable</u>		<u>Planificación Intercalada Seriarizable</u>		<u>Planificación Intercalada No Seriarizable</u>	
<u>Tiempo</u>	T1	T2	T1	T2	T1	T2
t ₁	BEGIN		BEGIN		BEGIN	
t ₂	READ(A)		READ(A)		READ(A)	
t ₃	A=A-500		A=A-500		A=A-500	
t ₄	WRITE(A)		WRITE(A)			BEGIN
t ₅	READ(B)			BEGIN		READ(A)
t ₆	B=B+500			READ(A)		AUX=A*0.1
t ₇	WRITE(B)			AUX=A*0.1		A=A-AUX
t ₈	COMMIT			A=A-AUX		WRITE(A)
t ₉		BEGIN		WRITE(A)		READ(B)
t ₁₀		READ(A)			WRITE(A)	
t ₁₁		AUX=A*0.1	READ(B)		READ(B)	
t ₁₂		A=A-AUX	B=B+500		B=B+500	
t ₁₃		WRITE(A)	WRITE(B)		WRITE(B)	
t ₁₄		READ(B)	COMMIT		COMMIT	
t ₁₅		B=B+AUX		READ(B)		B=B+AUX
t ₁₆		WRITE(B)		B=B+AUX		WRITE(B)
t ₁₇		COMMIT		WRITE(B)		COMMIT
Valores inicales	Resultados	Ok	Resultados	Ok	Resultados	✗
A = 2000 B = 2500	A = 1350 B = 3150		A = 1350 B = 3150		A = 1800 B = 2700	

Planificación no serializable:
ya que los valores que resultan
para A y B no son los mismos
que en la planificación en serie

← INCORRECTO

CONTROL DE CONCURRENCIA

Mecanismos para evitar los problemas de concurrencia garantizando la serialización o secuencialidad de la planificación de transacciones:

- Bloqueos (**locks o candados**).
- Marcas de tiempo (**timestamps**).
- Técnicas de validación (**Optimista**).

CONTROL DE CONCURRENCIA

- Algunas de las principales técnicas que se utilizan para controlar la ejecución concurrente de transacciones están basadas en el concepto de bloqueo de elementos de datos.
- Un **bloqueo**, **candado** o **lock** es una variable asociada a un elemento de datos que describe el estado de ese elemento respecto a las posibles operaciones que se le puedan aplicar.
- Un lock o candado es una estructura que sólo puede ser adquirida por una hebra de ejecución (thread) a la vez.
- Generalmente, hay un bloqueo por cada elemento de datos de la base de datos. Los bloqueos se utilizan como un medio para sincronizar el acceso de las transacciones concurrentes a los elementos de la base de datos.

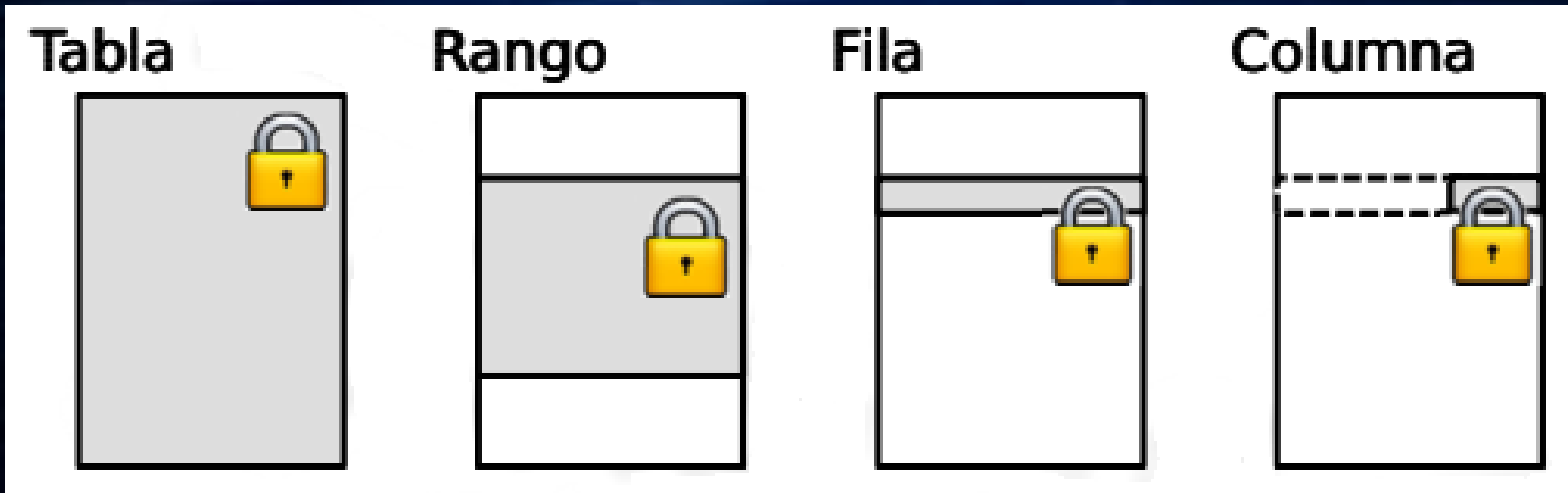
CONTROL DE CONCURRENCIA

- Una transacción coloca un lock o candado a un objeto de la base de datos para impedir que otra transacción lo modifique.
- Los locks o candados pueden tener distintas granularidades: base de dato, tabla, tupla o fila, atributo.
- Si dos ejecuciones tratan de obtener un lock o candado para actualizar una tabla, la primera que trate de obtenerlo tendrá acceso exclusivo a la tabla, la segunda debe esperar a que la primera lo suelte para obtener el acceso.
- Además de los locks o candados exclusivos existen locks o candados de sólo lectura y locks o candados compartidos que pueden estar simultáneamente siendo utilizados por distintas ejecuciones.

CONTROL DE CONCURRENCIA

Gránulo de bloqueo:

- Un gránulo es una unidad que puede ser bloqueada por una transacción.
- Dependiendo del SMD, el gránulo puede ser la base de datos, una tabla, fila, rango de filas, una columna de una fila, atributo, entre otras.
- Si una transacción intenta bloquear un gránulo bloqueado, espera hasta su liberación.



Tipos de bloqueo:

- **Para lectura (compartido):** un gránulo puede estar bloqueado para lectura por varias transacciones; los intentos de bloqueo exclusivo deben esperar.
- **Para escritura (exclusivo):** un gránulo puede estar bloqueado para escritura por una sola transacción; los demás intentos de bloqueo deben esperar.
- Matriz de compatibilidad de bloqueos:

	La transacción 2 solicita un candado compartido	La transacción 2 solicita un candado exclusivo
La transacción 1 no tiene candado	Sí	Sí
La transacción 1 tiene un candado compartido	Sí	No
La transacción 1 tiene un candado exclusivo	No	No

Bloqueo binario:

- Si una transacción quiere acceder a un dato debe bloquearlo antes.
- Cada transacción debe seguir las siguientes reglas:
 - Hacer LOCK(X) antes de operar sobre X.
 - Hacer UNLOCK(X) luego de operar sobre X.
 - No hacer LOCK(X) si ya tiene el bloqueo sobre X.
 - No hacer UNLOCK(X) a menos que tenga el bloqueo sobre X.
- No se permite interferir si hay un bloqueo sobre X (esperar a que sea liberado).

Bloqueo Multimodo:

- Admite bloqueos exclusivos para escritura (W-LOCK) y compartidos para lectura (R-LOCK) --> menos restrictivo.
- Cada transacción debe seguir las siguientes reglas:
 - Hacer R-LOCK(X) o W-LOCK(X) antes de leer X.
 - Hacer W-LOCK(X) antes de escribir X.
 - Hacer UNLOCK(X) luego de completar las operaciones sobre X.
 - No hacer R-LOCK(X) si ya tiene un R-LOCK(X).
 - No hacer W-LOCK(X) si ya tiene un W-LOCK(X).
 - No hacer un UNLOCK(X) a menos que tenga bloqueo de lectura o escritura sobre X.
- Variación: los bloqueos pueden ser promovidos (R-LOCK --> W-LOCK) o degradados (W-LOCK --> R-LOCK).

Garantizar la seriabilidad:

- El uso de bloqueos (binarios o multi-modo) por sí solo **NO GARANTIZA SERIABILIDAD** --> posibilidad de resultados incorrectos en la ejecución de transacciones.

Tiempo	T1	T2	T1	T2
t ₁	BEGIN			BEGIN
t ₂	READ(Y)			READ(Y)
t ₃	READ(X)			READ(X)
t ₄	X= X+Y			Y= Y+X
t ₅	WRITE(X)			WRITE(Y)
t ₆	COMMIT			COMMIT
t ₇		BEGIN	BEGIN	
t ₈		READ(Y)	READ(Y)	
t ₉		READ(X)	READ(X)	
t ₁₀		Y= Y+X	X= X+Y	
t ₁₁		WRITE(Y)	WRITE(X)	
t ₁₂		COMMIT	COMMIT	

Tiempo	T1	T2
t ₁	BEGIN	
t ₂	R-LOCK(Y)	
t ₃	READ(Y)	
t ₄	UNLOCK(Y)	
t ₅		BEGIN
t ₆		R-LOCK(X)
t ₇		READ(X)
t ₈		UNLOCK(X)
t ₉		R-LOCK(Y)
t ₁₀		READ(Y)
t ₁₁		Y= Y+X
t ₁₂		WRITE(Y)
t ₁₃		UNLOCK(Y)
t ₁₄		COMMIT
t ₁₅	R-LOCK(X)	
t ₁₆	READ(X)	
t ₁₇	X= X+Y	
t ₁₈	WRITE(X)	
t ₁₉	UNLOCK(X)	
t ₂₀	COMMIT	

Valores Iniciales: X=2, Y=3

Resultado de los planes serie:

Si T1 --> T2: X= 5; Y= 8

Si T2 --> T1: X= 7; Y= 5

Resulatdo plan serialable:

X=5; Y= 5 --> NO SERIALIZABLE

Causa: ítems desbloqueados antes de tiempo.

CONTROL DE CONCURRENCIA

Protocolo de bloqueo de dos fases:

- Cada transacción debe seguir un cierto protocolo sobre la ubicación de los bloqueos y desbloques dentro de las transacciones.
- Este protocolo --> GARANTIZA SERIABILIDAD y no se requiere verificarlo.
- Las fases son:
 1. **Fase de expansión o de crecimiento:** puede adquirir bloqueos sobre los datos que necesita, pero no puede liberar ninguno.
 2. **Fase de contracción o decrecimiento:** puede liberar los datos pero no puede adquirir nuevos bloqueos (P2F Básico).
- TODOS los bloqueos preceden al primer desbloqueo.

CONTROL DE CONCURRENCIA

Protocolo de bloqueo de dos fases - ejemplo:

- Observe que ambas transacciones necesitaron un candado exclusivo sobre BAL, puesto que las dos planeaban actualizarla. La transacción de Jack solicitó y obtuvo primero el candado, y lo mantuvo hasta que terminó la modificación de BAL. Después liberó el candado y la transacción de Jill, que había estado suspendida (en estado de espera) mientras esperaba que el candado se liberara, pudo lograr éste y concluir.

HORA	Transacción de Jack	Transacción de Jill	BAL
t1	COMIENZA TRANSACCIÓN		
t2	Xlock BAL		
t3	read BAL (1000)	COMIENZA TRANSACCIÓN	1000
t4	BAL = BAL – 50 (950)	request Xlock Bal	1000
t5	write BAL (950)	WAIT	950
t6	COMPROMETIDA/DESBLOQUEA BAL	WAIT	950
t7		grant XlockBAL read BAL (950)	950
t8		BAL = BAL +100	950
t9		write BAL (1050)	1050
t10		COMPROMETIDA/DESBLOQUEA BAL	1050

Problema de la anulación en cascada:

- Surge al utilizar el protocolo de bloqueo de dos fases, cuando una transacción no comprometida que haya liberado sus candados sea deshecha. Si a una segunda transacción se le permitió leer un valor escrito por la transacción deshecha, también debe ser deshecha ya que leyó datos sucios. Este problema se soluciona con las siguientes variaciones del protocolo de bloqueo de dos fases.
- **Estricto:** requiere que las transacciones conserven sus candados exclusivos hasta que queden comprometidas (COMMIT), lo que impide las anulaciones en cascada.
- **Riguroso:** requiere que las transacciones mantengan todos sus candados, tanto compartidos como exclusivos, hasta que se comprometan.

CONTROL DE CONCURRENCIA

Problema de la anulación en cascada:

- **Escalamiento de bloqueo:** una transacción puede solicitar al principio candados compartidos que permitirán a otras transacciones concurrentes de sólo lectura, acceder a los ítem. Después, cuando la transacción esté lista para una actualización, solicita que el candado compartido se escale a un candado exclusivo.

El escalamiento puede tener lugar sólo durante la fase de crecimiento y tal vez requiera que la transacción espere hasta que otra transacción libere un candado compartido sobre el ítem. Una vez que el ítem ha sido actualizado, su candado puede ser disminuido de uno exclusivo a uno compartido. La disminución puede tener lugar sólo durante la fase de decrecimiento.

CONTROL DE CONCURRENCIA

Problema de la anulación en cascada - ejemplo:

- La transacción R bloqueo a “a”, leyó su valor, lo actualizo y liberó su candado. Se le asigno el bloqueo sobre “a” a la transacción S leyó su valor, lo actualizo y liberó su candado. Se le asigno el bloqueo sobre “a” a la transacción T leyó su valor, lo actualizo y liberó su candado. Cuando R se deshizo, las operaciones sobre “a” se invalidaron, lo que hizo que S se deshiciera e invalidara las operaciones sobre “a”, por lo tanto T también se deshizo e invalido las operaciones sobre “a”.

Hora	R	S	T	a
t1	COMIENZA TRANS			1
t2	Bloqueo X de a	COMIENZA TRANS		
t3	read a (1)	solicitud de bloqueo X de a	COMIENZA TRANS	
t4	a=a+3 (4)	en espera	solicitud de bloqueo X de a	
t5	write a (4)	en espera	en espera	4
t6	unlock a	en espera	en espera	
		solicitud de bloqueo X de a		
t7		read a(4)	en espera	
t8		a = a*2 (8)	en espera	
t9		write a(8)	en espera	8
t10		desbloquear a	en espera	
			garantizado el	
t11			bloqueo X de a (8)	
t12			a=a+10 (18)	
t13			write a (18)	18
t14	DESHECHA			
t15		DESHECHA		
t16			DESHECHA	

Problema del interbloqueo (**deadlock**):

- Conocido también como candado mortal. Se produce cuando cada transacción T_i en un conjunto de dos o más transacciones está esperando a algún elemento que está bloqueado por alguna otra transacción T_j de dicho conjunto. Por tanto, cada transacción del conjunto está parada en espera a que una de las demás transacciones del conjunto libere el bloqueo sobre un elemento.
- **Ejemplo:** La figura muestra dos transacciones, S y T, que tienen un interbloqueo; la razón es que cada una espera que la otra libere un bloqueo de un ítem que maneja.

Hora	Transacción S	Transacción T
t1	Bloqueo X de a	. . .
t2	. . .	bloqueo X de b
t3	solicitud de bloqueo X de b	. . .
t4	en espera	solicitud de bloqueo S de a
t5	en espera	en espera
t6	en espera	en espera
t7	en espera	en espera
. . .	. . .	. . .

CONTROL DE CONCURRENCIA

Problema del interbloqueo (**deadlock**) - ejemplo:

- La figura ilustra cuatro transacciones, Q, R, S y T, que tienen un candado mortal porque Q espera un ítem bloqueado por R, R espera un ítem de datos bloqueado por S, S espera un ítem bloqueado por T, y T espera un ítem bloqueado por Q.

Hora	Trans Q	Trans R	Trans S	Trans T
t1	bloqueo X por Q1	. . .	. . .	. . .
t2	. . .	bloqueo X por R1	. . .	. . .
t3	. . .	. . .	bloqueo X por S1	. . .
t4	. . .	. . .	. . .	bloqueo X por T1
t5	solicitud de bloqueo R1	. . .	. . .	. . .
t6	en espera	solicitud de bloqueo S1	. . .	. . .
t7	en espera	en espera	solicitud de bloqueo T1	. . .
t8	en espera	en espera	en espera	solicitud de bloqueo Q1
t9	en espera	en espera	en espera	en espera
. . .	. . .	. . .	. . .	. . .

CONTROL DE CONCURRENCIA

Técnicas generales para manejar los interbloqueos:

- Una vez que ocurre un candado mortal, las aplicaciones involucradas no son capaces de resolver el problema. En vez de ello, el sistema tiene que reconocer la existencia del candado mortal y romperlo de algún modo.
- **Impedirlos:** que es cuando el sistema busca hacia delante si una transacción ocasionaría un interbloqueo y nunca permitiría que éste ocurriera.
- **Detección y recuperación:** permite que suceda el interbloqueo pero, una vez ocurrido, lo detecta y lo rompe. Como es más fácil determinar si hay un interbloqueo para romperlo, que evitar que ocurra, muchos sistemas emplean el método de detección y recuperación.

CONTROL DE CONCURRENCIA

Técnica de detección y recuperación de interbloqueos:

- Los esquemas de detección de interbloqueos utilizan un grafo de solicitud de recursos o grafo de espera para mostrar cuáles son las transacciones a la espera de los recursos bloqueados por otras.
- En dicho grafo, cada nodo representa una transacción. Un arco indica que una transacción está a la espera de que otra libere un bloqueo.
- El grafo de espera es mantenido por el sistema. Nuevos nodos se agregan al grafo a medida que comienzan nuevas transacciones.
- Cuando una transacción hace la solicitud de un bloqueo sobre cierto recurso bloqueado, se agrega un arco al grafo. Si el recurso es desbloqueado y obtenido por la transacción, el arco se borra.
- Cuando se presenta un interbloqueo, el grafo contendrá un ciclo.

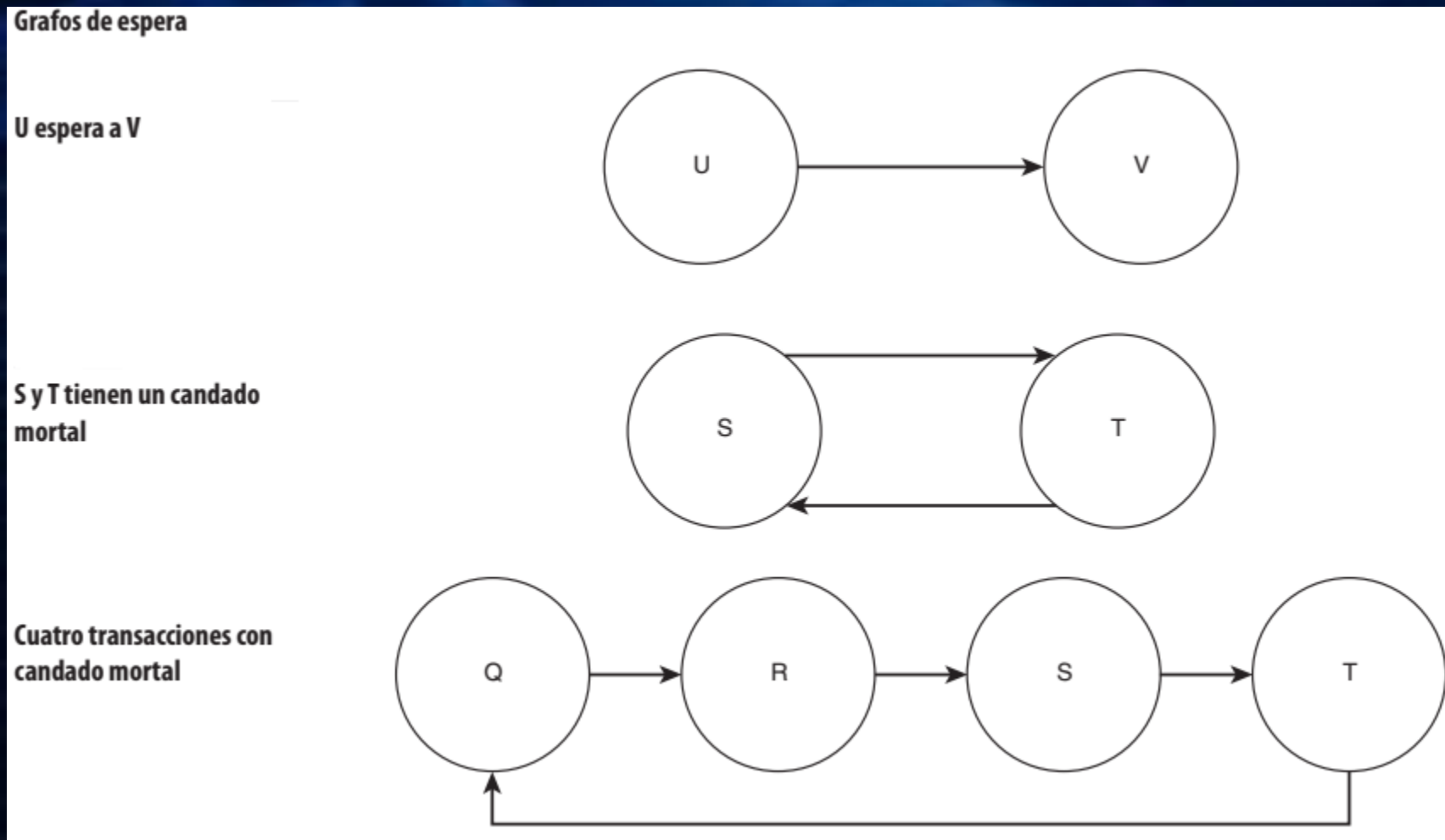
Técnica de detección y recuperación de interbloqueos:

- El sistema detecta interbloqueos por medio de la revisión periódica de su grafo de espera, en busca de ciclos.
- Debe revisar ciclos de cualquier longitud, ya que uno puede involucrar muchas transacciones. Una vez que se detecta un ciclo, el sistema debe resolver el interbloqueo.
- Lo hace por medio de escoger a una de las transacciones del ciclo como víctima, la cual tendrá que ser deshecha (rollback), liberando de esta manera los bloqueos que tenía y permitiendo que las demás transacciones que están en el ciclo puedan continuar.
- La víctima puede ser la transacción más nueva, que no ha realizado mucho trabajo, o la que haya hecho menos actualizaciones, entre otras opciones.
- Se debe tenerse el cuidado de evitar elegir siempre a la misma transacción como la víctima, situación que se llama **inanición o livelock**, ya que esa transacción nunca terminará.

CONTROL DE CONCURRENCIA

Técnica de detección y recuperación de interbloqueos:

- Ejemplos de grafos de espera, los dos últimos corresponden a los dos ejemplos anteriores de interbloqueos, como se ve presentan ciclos.



- Este protocolo no utiliza bloqueos, por lo que no pueden presentarse interbloqueos. Además garantiza que la ejecución será serializable.
- Una marca de tiempo (**timestamp**) para una transacción es un identificador único que indica el momento relativo de inicio de la transacción.
- Puede ser la lectura del reloj interno en el momento en que comienza la transacción, o el valor de un contador lógico que se incrementa cada vez que inicia una transacción nueva.
- El valor de la marca de tiempo de cada transacción T , que se denotará con $TS(T)$, es único e indica qué tan antigua es la transacción.
- El efecto de una marca de tiempo es asignar un orden serial a las transacciones.

- El protocolo asegura que el orden se cumpla, no permitiendo ninguna lectura o escritura que viole dicho orden.
- Además de las marcas de tiempo para las transacciones, las hay para ítem de datos.
- Cada ítem de datos, X , contiene dos marcas de tiempo:
 - **Marca de tiempo de lectura(X) o read(X):** que da la marca de tiempo de la última transacción que leyó los ítem. Se actualiza cuando una transacción realiza una operación de lectura (X).
 - **Marca de tiempo de escritura(X) o write(X):** que da la marca de tiempo de la última transacción que escribió los ítem. Se actualiza cuando se efectúa una operación de escritura(X).

Protocolo básico de marcas de tiempo:

- Este protocolo garantiza que las transacciones sean seriables y los resultados son equivalentes a una secuencia de ejecución serial en la que las transacciones son ejecutadas en orden cronológico por las marcas de tiempo.
- Problemas que pueden surgir cuando se utilizan marcas de tiempo:
 1. **Lectura tardía:** una transacción T, pide leer un ítem que ya ha sido actualizado por una transacción más reciente. Por lo que el valor que T necesita leer ya no se encuentra disponible debido a que fue sobrescrito, entonces T debe deshacerse y volver a comenzar con una marca de tiempo posterior.
 2. **Escritura tardía:** una transacción T, pide escribir un ítem cuyo valor actual ya fue leído o sobre escrito por una transacción más reciente. Por lo que otra vez, la solución está en deshacer T y reiniciarla con una marca de tiempo posterior.

Protocolo básico de marcas de tiempo - reglas:

Siempre se prueba la marca de tiempo de la transacción $TS(T)$, contra la marca de tiempo de escritura(X) y/o la marca de tiempo de lectura(X) que identifica la(s) transacción(es) que escribieron o leyeron los ítem de datos por última vez.

1. Si T pide leer un ítem de datos X , se compara su $TS(T)$ con la marca de tiempo de escritura(X).
 - A. Si la estampa de tiempo de escritura(X) $\leq TS(T)$, entonces se procede a usar el valor actual de los datos y se establece el valor de la marca de tiempo de lectura(X) con $TS(T)$. Sin embargo, si la marca de tiempo de lectura aún tiene el valor más grande que $TS(T)$ sólo la lee y no cambia la marca de tiempo de lectura(X).
 - B. Si la estampa de tiempo de escritura(X) $> TS(T)$, entonces T llega tarde para hacer su lectura, y el valor de X que necesita ya fue sobrescrito, por lo que se deshace T .
2. Si T pide escribir un ítem de datos X , se compara $TS(T)$ tanto con la marca de tiempo de escritura(X) como con la marca de tiempo de lectura(X).
 - A. Si la marca de tiempo de escritura(X) $\leq TS(T)$ y la marca de tiempo de lectura(X) $\leq TS(T)$, entonces se escribe y reemplaza la marca de tiempo de escritura(X) con $TS(T)$.
 - B. En otro caso, se deshace T , se asigna una nueva marca de tiempo y se reinicia T .

Protocolo básico de marcas de tiempo - ejemplo:

El siguiente ejemplo resuelve el problema del análisis inconsistente, presentado anteriormente.

Hora	SUMBAL	TRANSFER	BAL_A,R,W	BAL_B,R,W	BAL_C,R,W	SUM
t1	BEGIN TRANSACTION- Assign timestamp t1		5000,t0,t0	5000,t0,t0	5000,t0,t0	—
t2	SUM = 0;	BEGIN TRANSACTION- Assign timestamp t2	5000,t0,t0	5000,t0,t0	5000,t0,t0	—
t3	read BAL_A (5000)	. . .	5000,t1,t0	5000,t0,t0	5000,t0,t0	—
t4	SUM = SUM + BAL_A (5000)	read BAL_A (5000)	5000,t2,t0	5000,t0,t0	5000,t0,t0	—
t5	read BAL_B (5000)	BAL_A = BAL_A – 1000 (4000)	5000,t2,t0	5000,t1,t0	5000,t0,t0	—
t6	SUM = SUM + BAL_B (10000)	write BAL_A (4000)	4000,t2,t2	5000,t1,t0	5000,t0,t0	—
t7		read BAL_C (5000)	4000,t2,t2	5000,t1,t0	5000,t2,t0	—
t8		BAL_C = BAL_C + 1000 (6000)	4000,t2,t2	5000,t1,t0	5000,t2,t0	—
t9		write BAL_C (6000)	4000,t2,t2	5000,t1,t0	6000,t2,t2	—
t10	Read BAL_C –lectura tardía DESHACER		4000,t2,t2	5000,t0,t0	6000,t2,t2	—
t11		COMMIT				
t12	restart with timestamp t12					
. . .						

CONTROL DE CONCURRENCIA

- Las técnicas de validación, también llamadas **optimistas**, funcionan con la suposición de que normalmente no habrá un conflicto.
- Permiten que las transacciones se efectúen como si no existieran problemas de concurrencia, pero justo antes de que una transacción se comprometa, se revisa para determinar si ha ocurrido un conflicto. Si lo hubiera, la transacción debe deshacerse. Como la suposición es que es raro que el conflicto suceda, el deshacer una transacción será poco común. El deshacer de modo ocasional es el precio que se paga por eliminar los bloqueos.
- Estas técnicas permiten más concurrencia que los protocolos o técnicas tradicionales porque no se realizan bloqueos.

CONTROL DE CONCURRENCIA

- Durante la ejecución de la transacción, todas las actualizaciones se aplican a copias locales de los elementos de datos que se conservan para la transacción.
- Al término de la ejecución de la transacción, una fase de validación comprueba si alguna de las actualizaciones de la transacción viola la serialización.
- El sistema debe conservar cierta información necesaria para la fase de validación. Si la serialización no se viola, la transacción es confirmada y se actualiza la base de datos a partir de las copias locales; en caso contrario, la transacción es abortada y reiniciada posteriormente.

Fases del protocolo:

1. **Fase de lectura:** que se extiende del principio de la transacción hasta apenas antes de que se comprometa. Durante esta fase la transacción lee los valores de todas las variables que necesita, y los guarda en variables locales. Si la transacción hace cualesquiera escrituras, éstas se hacen a la copia local de los datos, no a la base de datos en sí.
2. **Fase de validación:** en este momento, la transacción prueba a fin de determinar si hay alguna interferencia.

Para una transacción de sólo lectura, esto consiste en observar que no haya errores causados por otra transacción que hubiera estado activa cuando se leyeron los valores de los datos. Si no hubo error, la transacción se compromete. Si ocurrió una interferencia, la transacción se aborta y reinicia.

Para una transacción que hace actualizaciones, la validación consiste en determinar si la transacción actual dejará la base de datos en un estado consistente, mediante seriabilidad. Si no es así, se aborta la transacción. Si la transacción de actualizar aprueba la validación, pasa a la tercera fase.

3. **Fase de escritura:** si la fase de validación es satisfactoria, las actualizaciones de la transacción son aplicadas a la base de datos; en caso contrario, las actualizaciones se descartan y se reinicia la transacción.